INFERENCE-TIME GUARDRAILS FOR MITIGATING PROMPT JAILBREAK ATTACKS

Final Report for Data Science & AI Lab Project
Submitted by Team No : 10

Name	Email
Jash Vora	23f1002016@ds.study.iitm.ac.in
Kondaveeti Muralidhar	22f3000528@ds.study.iitm.ac.in
Anirudh Narayan D	21f1001877@ds.study.iitm.ac.in
C S Sruthi	21f1006510@ds.study.iitm.ac.in
Vasanth Bupathi S	23f1002451@ds.study.iitm.ac.in

IITM Online BS Degree Program,

Indian Institute of Technology, Madras, Chennai

Tamil Nadu, India, 600036

ABSTRACT

Large Language Models (LLMs) are increasingly deployed in safety-critical production environments such as customer support automation, enterprise copilots, and decision-support systems. Despite advances in training-time alignment techniques, including Reinforcement Learning from Human Feedback and Constitutional AI, these models remain highly vulnerable to adversarial prompt jailbreak attacks. Such attacks exploit instruction-following behavior through role-play framing, instruction overrides, and prompt injection, enabling the generation of harmful or policy-violating content.

This project presents a production-ready, inference-time guardrail system that mitigates prompt jailbreak attacks without modifying or retraining the underlying Large Language Model. The proposed solution is implemented as middleware between the user interface and the Gemini API, employing a hybrid defense-in-depth architecture that combines deterministic rule-based filtering, a fine-tuned mDeBERTa-v3-base semantic classifier with mean pooling, and a tri-modal threshold-based decision engine. Prompts classified as high-risk are blocked immediately, low-risk prompts are allowed, and moderately suspicious prompts are optionally sanitized using an external LLM prior to re-evaluation.

The system is evaluated on a 20,137-sample benchmark corpus using standardized metrics including Attack Success Rate, False Refusal Rate, macro-F1 score, and latency overhead. On a held-out test set, the full pipeline achieves an Attack Success Rate of 1.77%, a False Refusal Rate of 3.94%, and a composite safety-utility score of 0.9656, with an average inference latency of 5.84 milliseconds per request on a single NVIDIA T4 GPU. These results demonstrate that robust jailbreak mitigation can be achieved through modular inference-time defenses that satisfy real-world latency and deployment constraints.

1. Introduction

Large Language Models (LLMs) have become foundational components of modern software systems, enabling natural-language interfaces for information retrieval, customer support, content generation, and enterprise automation. As these models transition from experimental deployments to large-scale production use, ensuring their safe and reliable operation has emerged as a critical challenge. Even highly aligned models trained using techniques such as Reinforcement Learning from Human Feedback (RLHF) remain susceptible to adversarial prompt manipulation.

Prompt jailbreak attacks exploit the instruction-following nature of LLMs using techniques such as role-play hijacking, instruction overrides, and prompt injection. These attacks can bypass safety constraints embedded within the model, resulting in harmful, unethical, or policy-violating outputs. In production environments, such failures introduce significant legal, compliance, reputational, and operational risks.

Most existing defenses are implemented at training time and embedded directly within the model's parameters. While effective against common failure cases, these approaches are inherently static and difficult to update after deployment. This limitation is particularly pronounced in black-box API settings, where retraining or internal policy modification is infeasible. Lightweight mitigations such as system prompt hardening or keyword-based filtering provide limited robustness against adaptive adversaries and often degrade user experience through excessive false refusals.

To address these limitations, this project proposes an inference-time guardrail architecture that operates independently of the base model. Implemented as middleware between the user application and the Gemini API, the system integrates multiple complementary safeguards: a deterministic regex-based pre-filter for high-confidence attack signatures, a low-latency semantic classifier based on mDeBERTa-v3-base with attention-aware mean pooling, and a calibrated threshold-based decision layer that optionally invokes LLM-based prompt transformation. By explicitly balancing safety, task utility, and latency, the proposed system is designed for practical deployment in real-world LLM applications.

2. Literature Review

This section reviews prior academic research and industry-deployed mechanisms for mitigating prompt jailbreak attacks and improving LLM safety. The

objective is to contextualize the proposed approach within existing work and to identify gaps addressed by the final system architecture.

2.1 Training-Time Alignment Approaches

Reinforcement Learning from Human Feedback (RLHF)

RLHF aligns LLM outputs with human preferences by optimizing model behavior using a learned reward function. Modern models such as GPT-4, Claude, and Gemini rely heavily on this paradigm.

Strengths

- Safety behaviors embedded directly into model weights
- No inference-time latency overhead
- Strong performance on cooperative benchmarks

Limitations

- Vulnerability to adversarial prompt manipulation
- Static safety policies after deployment
- Costly retraining requirements
- Ineffectiveness in closed-source API environments

Empirical benchmarks such as JailbreakBench report Attack Success Rates exceeding 60–80% for RLHF-aligned models under structured jailbreak attacks, indicating that training-time alignment alone is insufficient under adversarial input distributions.

Constitutional AI

Constitutional AI augments training with self-critique guided by predefined principles.

Strengths

- Scales alignment without extensive human labeling
- Improves refusal behavior under benign prompts

Limitations

- Susceptibility to instruction-override and role-play attacks
- Lack of explicit adversarial modeling
- Absence of post-generation verification

2.2 Prompt-Level and Rule-Based Defenses

System Prompt Hardening

Strengthening system prompts is a commonly deployed mitigation strategy.

Limitations

- Easily overridden through adversarial framing
- Absence of formal robustness guarantees

Keyword and Regex Filtering

Rule-based filtering approaches remain prevalent in production systems.

Strengths

- Near-zero inference latency
- Full interpretability

Limitations

- Poor robustness to paraphrasing
- High false refusal rates
- Lack of contextual understanding

Despite these limitations, deterministic filters remain valuable as first-line defenses for high-confidence attack signatures when paired with semantic classifiers.

2.3 LLM-as-a-Judge and Secondary-Model Approaches

Using a secondary large language model to assess safety can improve contextual reasoning.

Limitations

- High computational cost and latency
- Unsuitability for strict real-time deployment constraints

2.4 Small Specialized Safety Models

Recent work demonstrates the effectiveness of lightweight semantic classifiers for jailbreak detection. Meta's PromptGuard, based on mDeBERTa-v3, shows that encoder-only models can achieve strong detection performance with sub-10-millisecond inference latency.

Limitations

- Typically limited to input-side classification
- Lack of downstream output validation

2.5 Identified Gap

Existing defenses tend to optimize either safety at the expense of latency or latency at the expense of robustness. Few systems integrate deterministic rules, semantic classification, calibrated decision thresholds, and optional prompt transformation within a single production-ready inference-time pipeline.

3. Dataset

The dataset is explicitly engineered to support adversarial robustness evaluation, threshold calibration, and safety–utility trade-off analysis under realistic production conditions.

3.1 Dataset Design Objectives

The dataset was constructed to satisfy four primary objectives:

1. **Adversarial Robustness Evaluation:** The dataset must contain a sufficiently diverse and realistic distribution of jailbreak attacks to evaluate resistance against role-play framing, instruction overrides, prompt injection, and obfuscation.
2. **False Negative Minimization (Safety Priority)**: The dataset emphasizes adversarial coverage to stress-test the system's ability to detect safety-critical failures where malicious prompts bypass guardrails.
3. **False Refusal Control (Utility Preservation)**: A substantial benign subset is maintained to measure exaggerated safety behavior and ensure acceptable user experience.

4. **Statistically Meaningful Threshold Calibration**: The dataset size must be large enough to allow smooth probabilistic threshold sweeps for ASR–FRR trade-off tuning without overfitting to a small validation set.

3.2 Dataset Composition and Class Taxonomy

The final corpus consists of **20,137 unique prompts**, labeled into three canonical classes aligned with operational guardrail outcomes:

- **Benign**: Policy-compliant prompts that should be forwarded to the LLM without intervention.
- **Jailbreak**: Adversarial prompts whose intent is to bypass alignment constraints using structural manipulation rather than overtly malicious content.
- **Harmful**: Direct requests for unsafe, illegal, or prohibited actions independent of jailbreak framing.

3.2.1 Class Distribution

Split	**Total**	**Benign**	**Jailbreak**	**Harmful**
Train	14,093	5,683	6,761	1,649
Validation	3,017	1,217	1,447	353
Test	3,027	1,219	1,454	354
Total	**20,137**	**8,119**	**9,662**	**2,356**

Table 3.1 –Class Distribution

The dataset is intentionally **attack-heavy**, with jailbreak prompts forming the largest class. This distribution reflects real production risk, where adversarial prompts are rare but disproportionately costly when they succeed.

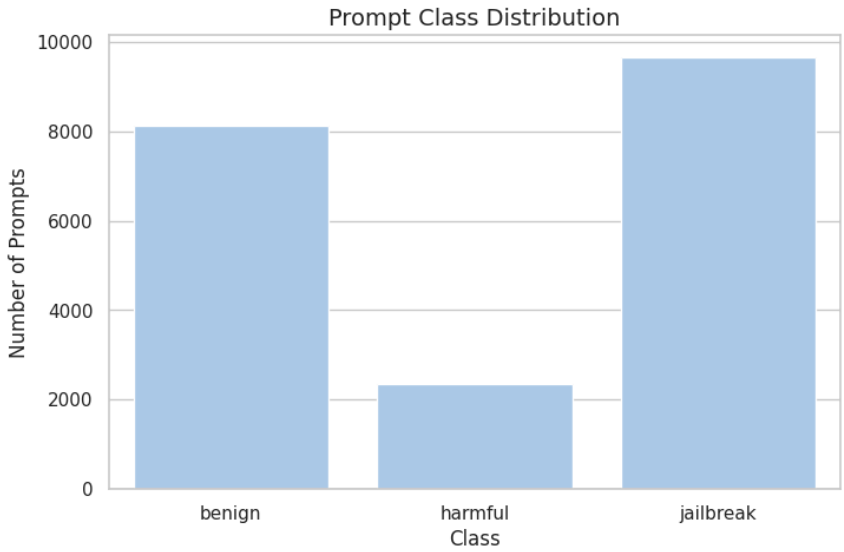

Figure 3.1 –Prompt Class Distribution

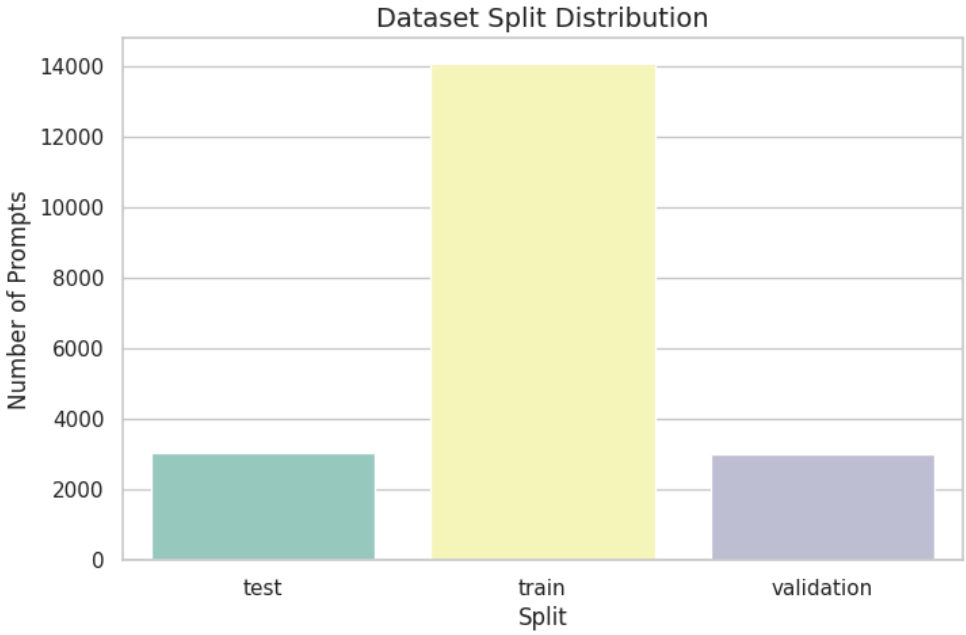

Figure 3.2 –Dataset Split Distribution

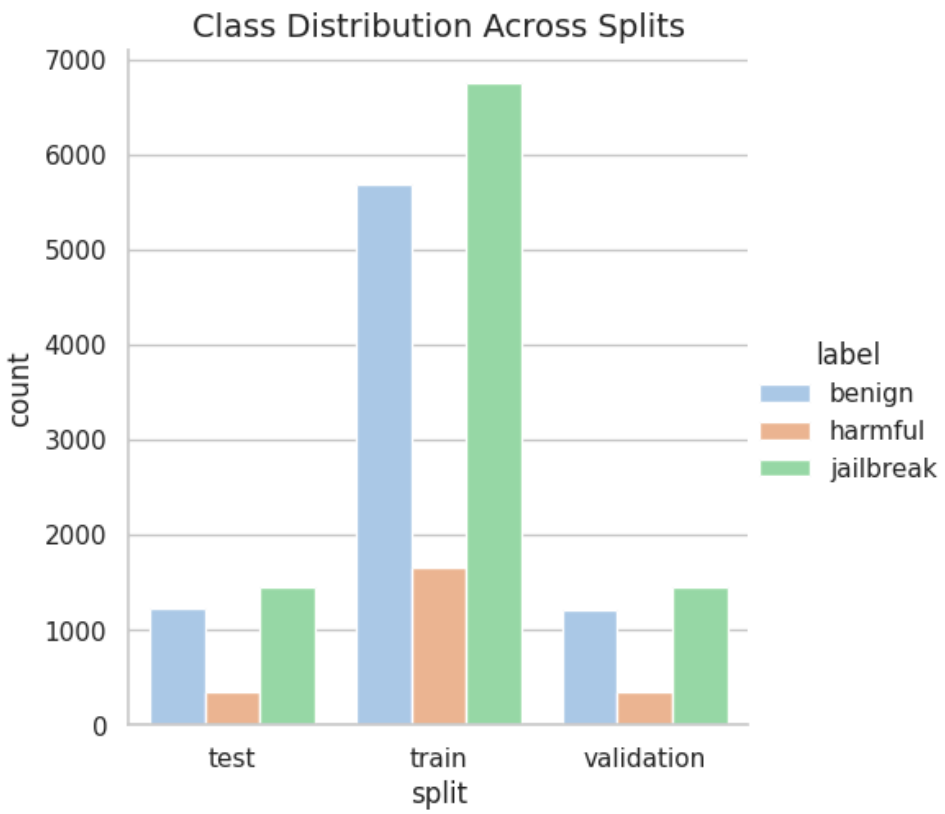

Figure 3.3 –Class Distribution across splits

3.3 Data Sources and Provenance

Prompts were sourced exclusively from publicly available, research-grade datasets to ensure legal use, reproducibility, and peer verifiability.

3.3.1 Benign Prompt Sources

- SQuAD v2 (question answering)
- Alpaca Cleaned (instruction-following prompts)
- TrustAIRLab In-The-Wild (benign partition)
- JailbreakBench benign behavior set

3.3.2 Jailbreak Prompt Sources

- TrustAIRLab In-The-Wild Jailbreak Prompts
- LMSYS Toxic-Chat jailbreak variants
- Public role-play jailbreak repositories

3.3.3 Harmful Prompt Sources

- JailbreakBench harmful directives
- LMSYS Toxic-Chat harmful conversations

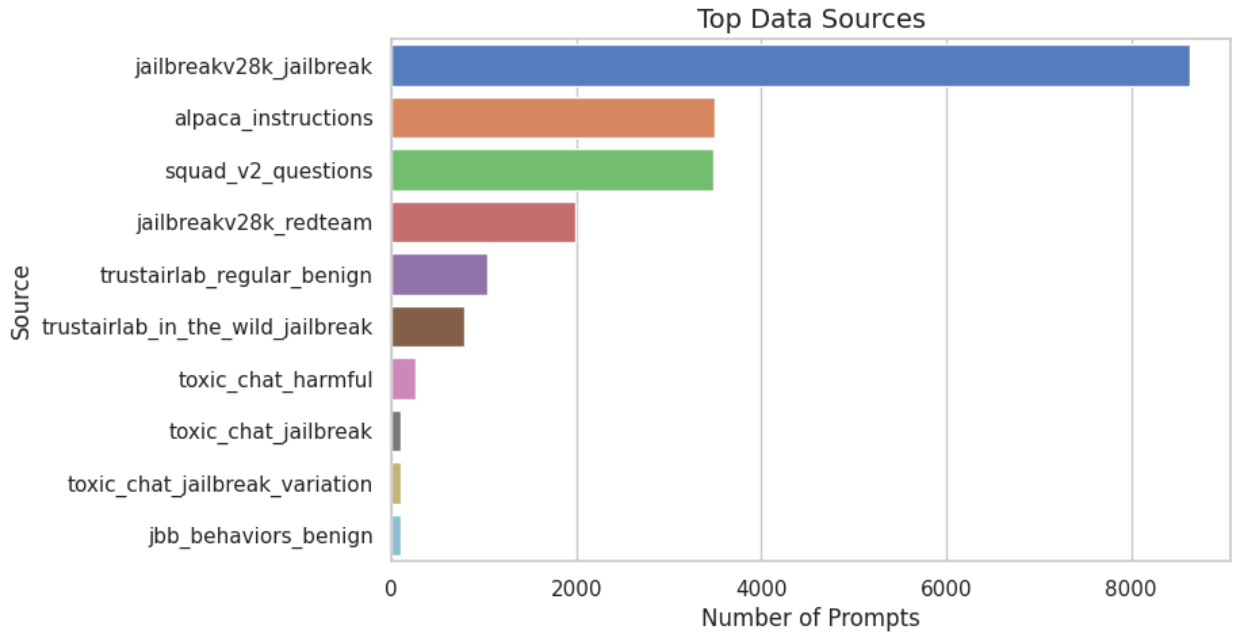

Figure 3.4 –Top Data Sources

3.4 Dataset Cleaning and Normalization Pipeline

To ensure high-quality supervision, raw prompts undergo a deterministic multi-stage preprocessing pipeline.

3.4.1 Schema Harmonization

Heterogeneous repository-specific fields (instruction, input, user_text, etc.) are mapped into a unified schema centered on a single prompt_text field. Malformed entries and null records are discarded during ingestion.

3.4.2 Structural Normalization

- Newlines, carriage returns, and tabs are collapsed
- Multiple whitespace sequences are reduced
- Leading and trailing whitespace is removed

This normalization standardizes token density without altering semantic intent.

3.4.3 Length Constraints

- Prompts shorter than **8 characters** are removed
- Upper-length enforcement ensures compatibility with transformer context limits

3.4.4 SHA-256 Deduplication

A stable SHA-256 hash is computed over normalized prompt text. Exact duplicates are removed globally, preventing redundancy across dataset sources and splits.

3.4.5 Ground-Truth Verification

Labels are cross-validated against original dataset metadata and manually audited in ambiguous cases to maintain label integrity.

3.5 Data Leakage Prevention via Family-Grouped Stratification

To prevent semantic data leakage between training, validation, and test sets, the dataset employs a **family-based grouping strategy**. Each prompt is assigned a deterministic family_id that represents a single semantic prompt family, encompassing the original prompt and any synthetic or paraphrased variants derived from it. The family-grouped stratification mechanism enforces the following constraints:

- Each base prompt is assigned a unique, deterministic family_id.
- All synthetically generated or paraphrased variants inherit the same family_id as their originating prompt.
- During dataset partitioning, all prompts sharing a family_id are assigned **exclusively to the same split** (train, validation, or test).

This design ensures that no semantic prompt family is ever observed across multiple splits, thereby guaranteeing that evaluation samples represent **previously unseen semantic content**, rather than paraphrased or structurally modified versions of training data.

3.5.1 Quantitative Verification of Leakage Prevention

Family-level leakage was verified both programmatically and statistically across the full dataset. The following results confirm the integrity of the split assignment:

- **Total unique prompt families:** 20,017
- **Prompt families spanning multiple splits:** 0
- **Family-level semantic leakage rate: 0.000000%**

The maximum number of splits occupied by any single family is **one**, establishing a strict upper bound that excludes any form of cross-split semantic overlap.

3.5.2 Prompt-to-Family Cardinality Analysis

Statistic	Value
Total families	20,017
Mean prompts per family	1.006
Standard deviation	0.077
Minimum	1
Median	1
75th percentile	1
Maximum	2

Table 3.2 –Prompt to Family Cardinality Analysis

These results indicate that the vast majority of families consist of a **single prompt**, with only a small fraction containing a limited number of syntactic variants. This confirms that family grouping is applied conservatively to prevent over-representation of any semantic intent while still allowing targeted augmentation for adversarial robustness testing.

3.5.3 Methodological Implications

The combination of family-grouped stratification and statistical verification provides a formal guarantee that:

- No semantic prompt family contaminates multiple splits.
- Evaluation metrics reflect genuine generalization performance.
- Measured Attack Success Rate (ASR) and False Refusal Rate (FRR) are not artificially deflated by memorization effects.

By enforcing strict semantic isolation at the family level, the dataset ensures that reported guardrail performance accurately represents behavior on **previously unseen**

adversarial strategies, rather than near-duplicate reformulations encountered during training.

3.6 Exploratory Dataset Characteristics

Exploratory Data Analysis reveals important structural properties that directly inform modeling decisions:

3.6.1 Prompt Length Distribution

- Jailbreak prompts are significantly longer (often >500 tokens)
- Benign prompts are short and direct
- Harmful prompts are concise but directive

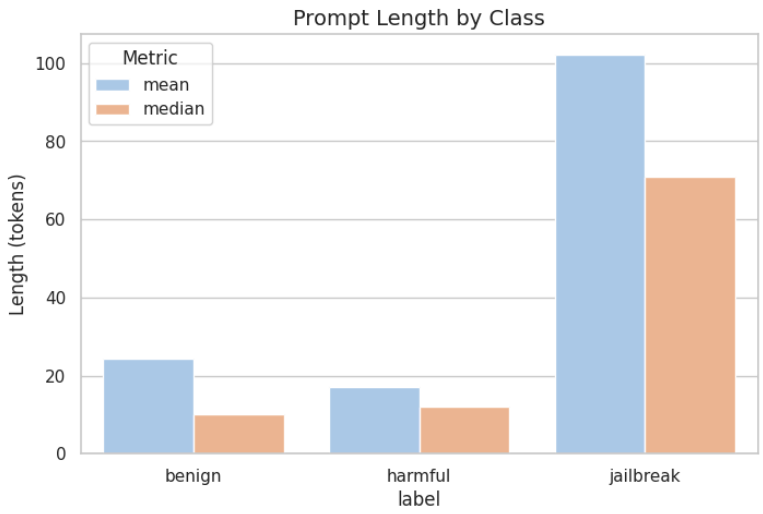

Figure 3.5– Prompt Length Distribution by Class

The long-tail distribution of jailbreak prompts motivates specialized truncation strategies discussed in the upcoming sections.

3.7 Jailbreak Taxonomy

Jailbreak prompts are annotated using a multi-dimensional taxonomy:

Category	Description
Role-Play	Persona adoption to bypass safety
Instruction Override	Explicit negation of safety rules

Multi-Step	Sequential decomposition of malicious intent
Prompt Injection	Structural priority manipulation
Obfuscation	Encoded or disguised malicious intent

Table 3.3 –Jailbreak Taxonomy

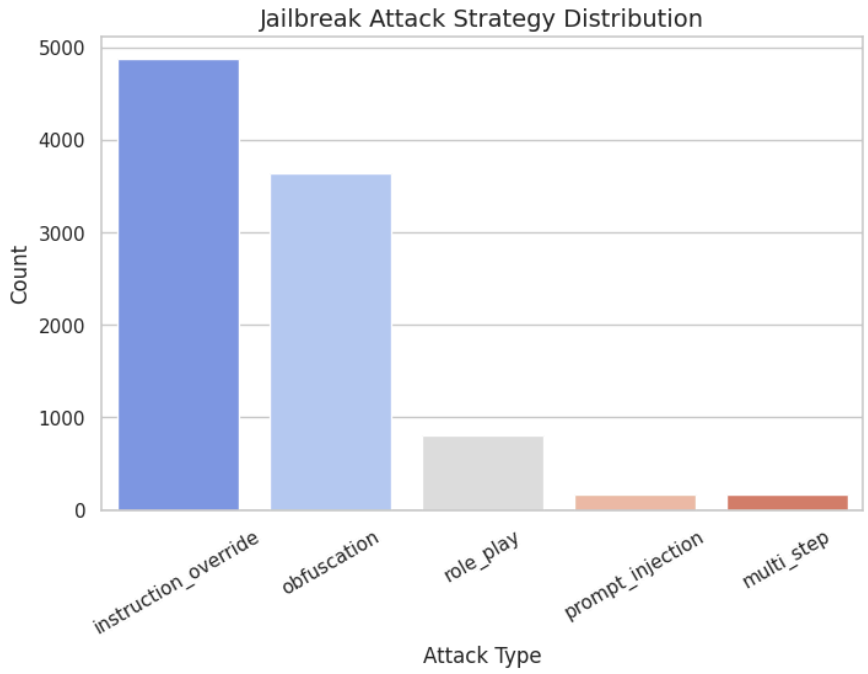

Figure 3.6 – Jailbreak Strategy Frequency

3.8 Tokenization and Truncation Strategy

Prompts are tokenized using the **mDeBERTa-v3 SentencePiece tokenizer**.

- Maximum context window: **512 tokens**
- Coverage: **98.35% of dataset**

3.8.1 Head-Tail Truncation

For prompts exceeding the limit:

- First half captures adversarial framing
- Second half captures terminal payload

This preserves bimodal attack structure and significantly outperforms left-only truncation on long-form attacks.

3.9 Ground-Truth Mapping to Guardrail Metrics

Operational metric mapping:

- **Benign blocked → FRR**
- **Jailbreak/Harmful allowed → ASR**

This mapping aligns offline evaluation metrics with real production guardrail failures.

4. Methodology

This section describes the end-to-end methodological framework of the proposed inference-time guardrail system. The focus is on **system architecture, inference-time control flow, and decision logic**, rather than dataset construction, which is detailed in Section 3.

4.1 Inference-Time Guardrail as Middleware

The guardrail is implemented as an **inference-time middleware** positioned between the user interface and the downstream Large Language Model (Gemini API). The base model is treated as a **black-box component**, enabling deployment without modification or retraining of the LLM.

Each incoming prompt is synchronously processed through a **hybrid defense-in-depth pipeline** before any LLM call is issued. This design enforces early rejection of high-risk prompts while preserving fast pass-through for benign queries.

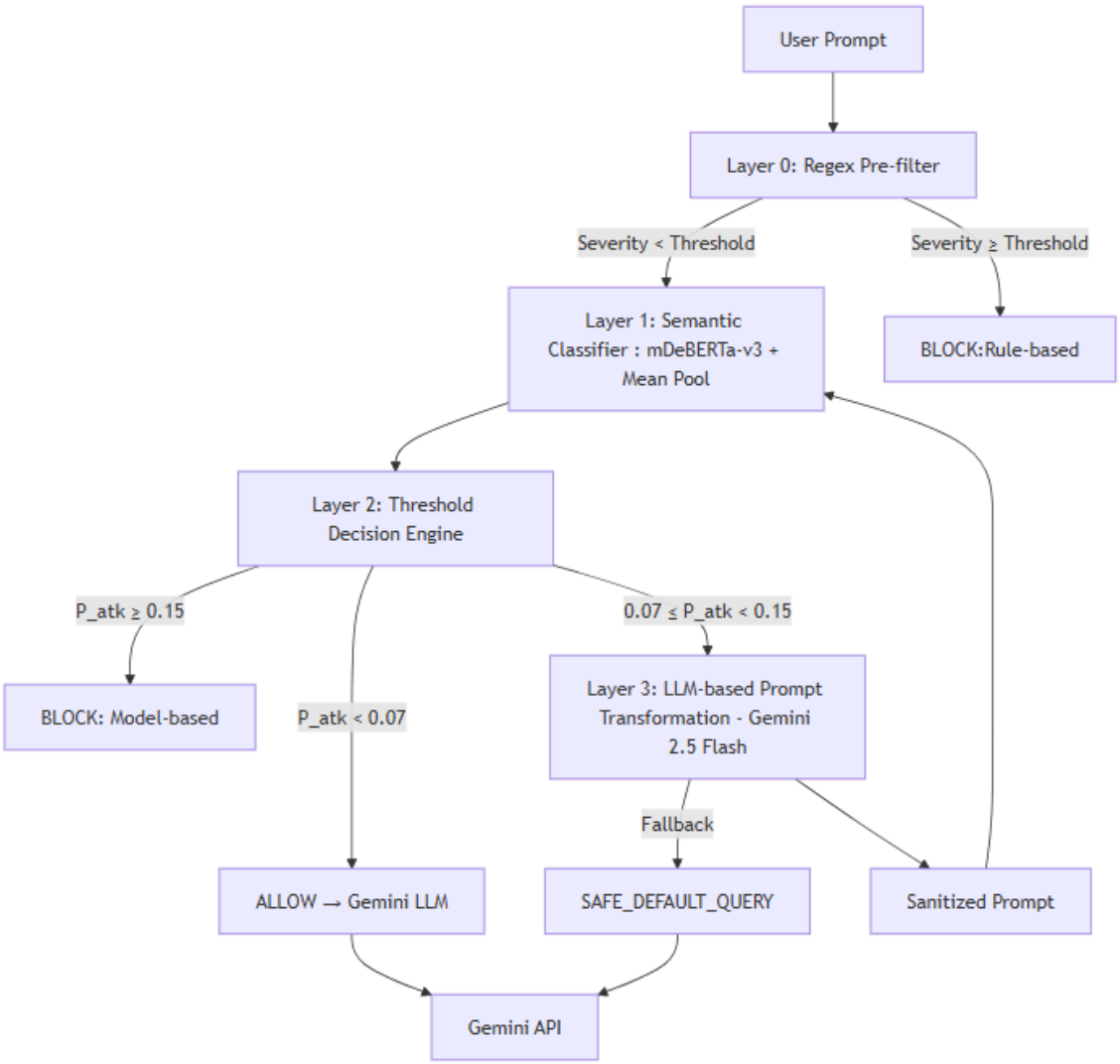

Figure 4.1 — Inference-Time Guardrail Middleware Architecture

4.2 Multi-Layer Defense-in-Depth Pipeline

Middleware applies a sequential, layered decision process:

1. **Layer 0 — Deterministic Regex Pre-Filter:** A severity-weighted regex engine scans normalized input text for high-confidence jailbreak or harmful patterns. Prompts whose cumulative severity exceeds a fixed threshold are blocked immediately without neural computation, providing sub-millisecond rejection and an interpretable audit trail.

2. **Layer 1 — Semantic Classification**: Prompts passing Layer 0 are evaluated by a fine-tuned mDeBERTa-v3-base classifier that models contextual adversarial intent beyond surface-level keyword matching.
3. **Layer 2 — Threshold-Based Decision Engine**: Classifier probabilities are mapped to explicit system actions using calibrated thresholds, routing prompts to **ALLOW**, **TRANSFORM**, or **BLOCK** states.
4. **Layer 3 — Optional LLM-Based Transformation**: Prompts in the intermediate risk range are sanitized via LLM-based rewriting and re-evaluated before being forwarded or blocked. A safe fallback ensures deterministic behavior under transformation failure.

This layered architecture prevents single-point failures and ensures robustness against both known and novel jailbreak strategies.

4.3 Tokenization and Long-Context Handling

All prompts are tokenized using the mDeBERTa-v3 SentencePiece tokenizer. A maximum sequence length of **512 tokens** is used, covering **98.35%** of prompts.

For the remaining long-form inputs, a **head–tail truncation strategy** is applied, preserving both adversarial framing (prefix) and terminal payload (suffix). This strategy is motivated by empirical analysis showing that jailbreak intent is bimodally distributed within long prompts.

4.4 Inference-Time Decision Logic

Let

$$P_{atk} = max(P(jailbreak), P(harmful))$$

System actions are determined as follows:

- **ALLOW** if $P_{atk} < 0.07$
- **TRANSFORM** if $0.07 \leq P_{atk} < 0.15$
- **BLOCK** if $P_{atk} \geq 0.15$

Thresholds are selected empirically on the validation set to optimize the safety–utility trade-off.

4.5 Evaluation Protocol

Evaluation is conducted exclusively on a held-out test set of unseen semantic prompt families. Performance is reported under two configurations:

- **Model-only** (semantic classifier + thresholds)
- **Full pipeline** (regex + classifier + thresholds + transformation)

Metrics include Attack Success Rate (ASR), False Refusal Rate (FRR), macro F1-score, and inference latency.

5. Model Development and Hyperparameter Tuning

This section describes the development of the semantic guardrail classifier, the training configuration, and the hyperparameter optimization process that led to the final production model. Architectural choices, truncation strategy, and calibration decisions are grounded in empirical validation results.

5.1 Model Architecture

The semantic classifier is built on the **mDeBERTa-v3-base encoder**, selected for its strong contextual modeling capabilities and suitability for low-latency, inference-time deployment. The model operates as a **three-class classifier** (benign, jailbreak, harmful) and forms the core detection component of the guardrail middleware.

Internally, the classifier employs **attention-mask-aware mean pooling** over token-level embeddings rather than relying on the [CLS] token representation. This architectural choice is motivated by the observation that adversarial intent in jailbreak prompts is often **distributed across the sequence**, particularly in long role-play or instruction-override attacks.

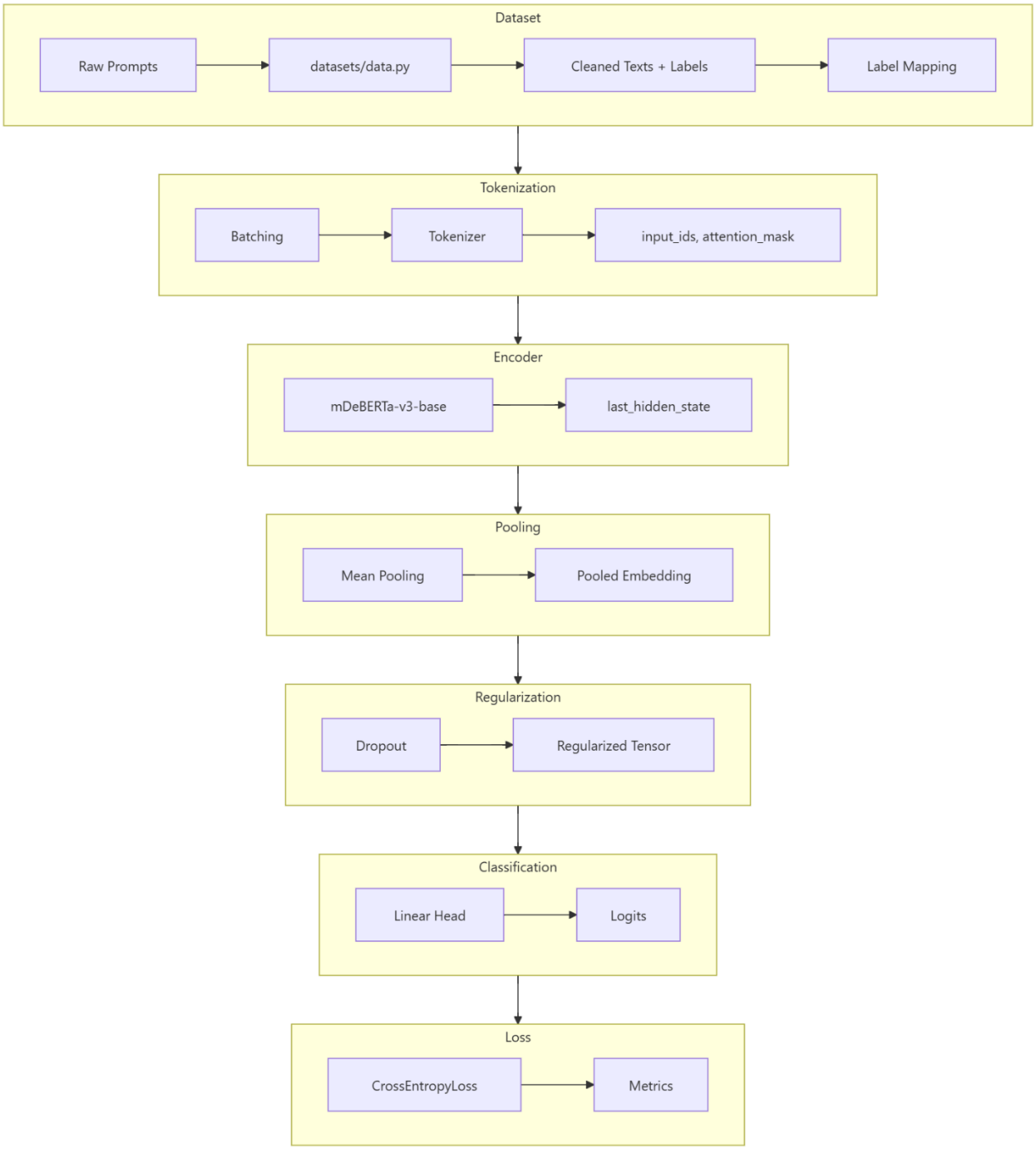

Figure 5.1 — Model Architecture

5.2 Training Configuration

Training is performed on the training split (N = 14,093) using supervised fine-tuning with AdamW optimization and early stopping.

Parameter	Value
MAX_LENGTH	512
Batch Size	4
Learning Rate	3e-5
Optimizer	Adam W
Weight Decay	0.01
Warmup Ratio	0.05
Dropout	0.2
Gradient Clipping	0.5
Max Epochs	10
Early Stopping Patience	3
Label Smoothing	0.0

Table 5.1 — Training Configuration

5.3 Loss Function and Calibration

The model is optimized using **cross-entropy loss with inverse-frequency class weighting** to address class imbalance, particularly the under-representation of the harmful class. Class weights are clamped to avoid instability.

Label smoothing ($\varepsilon = 0.1$) is applied to improve probability calibration. Calibration is critical because inference-time decisions rely on probabilistic thresholds rather than argmax classification. Without smoothing, softmax outputs collapse toward binary extremes, reducing the effective operating range for threshold tuning.

5.4 Hyperparameter Optimization

Hyperparameter selection is performed via a **99-trial random search**, exploring learning rate, dropout, maximum sequence length, and decision thresholds. Each trial is evaluated on the validation set (N = 3,017) using a composite objective combining F1, ASR, and FRR.

5.5 Maximum Sequence Length Selection

Although **MAX_LENGTH = 444** was locally optimal during tuning, the production model uses **MAX_LENGTH = 512**. The justification is shown in the sequence-coverage analysis.

Max Length	Coverage (%)	Samples Covered
444	94.92%	19,115
512	**98.35%**	**19,804**

Table 5.2— Sample Coverage based on Sequence Length

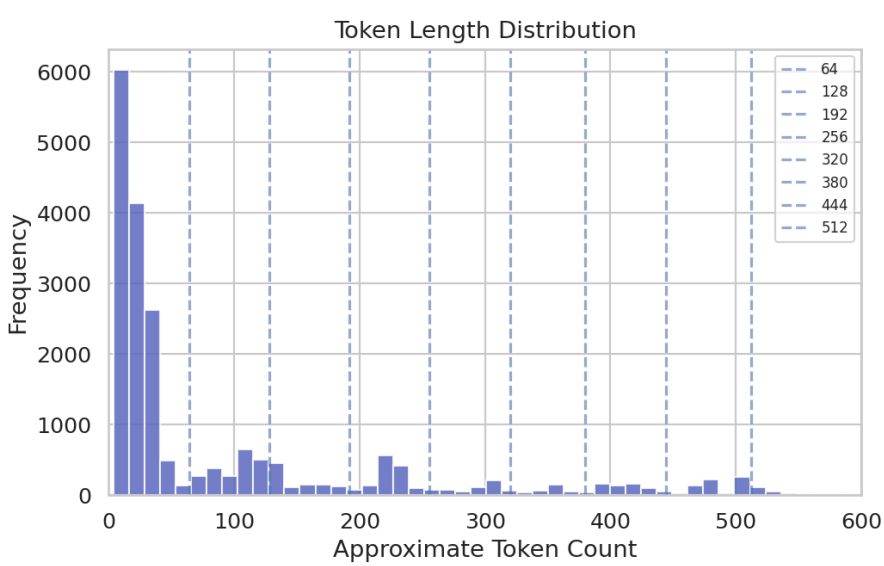

Figure 5.2 — Token Length Distribution

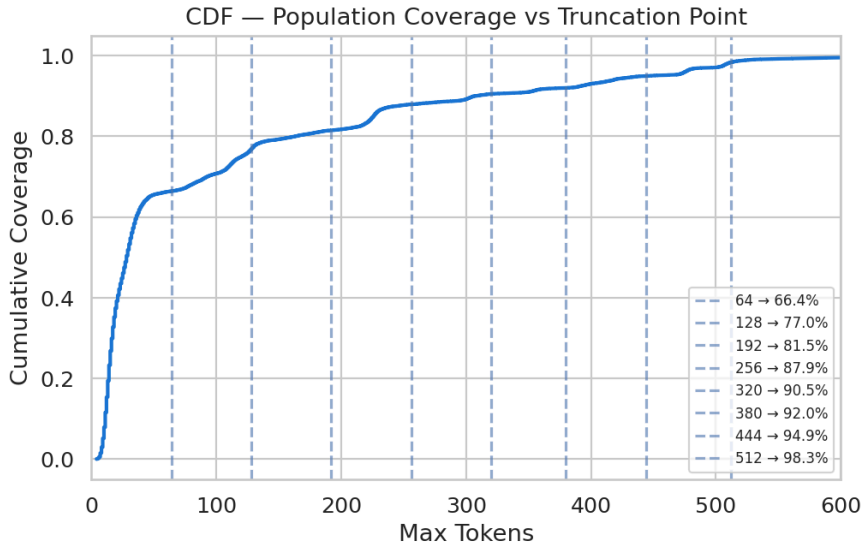

Figure 5.3 — Population Coverage vs Truncation Point

The additional coverage captures long-form adversarial prompts, and head–tail truncation preserves critical semantic content with minimal latency impact.

6 Evaluation & Analysis

This section presents the final evaluation of the guardrail system, including overall performance, confusion-matrix analysis, threshold sensitivity, error taxonomy, and confidence behavior. All results are reported on the **held-out test set (N = 3,027)** unless stated otherwise.

6,1 Evaluation Setup

Two configurations are evaluated:

1. **Model-Only:** Neural classifier with threshold-based decisions
2. **Model + Regex:** Regex pre-filter combined with neural classifier and thresholds

The test set consists of 1,219 benign, 1,454 jailbreak, and 354 harmful prompts, drawn from unseen semantic prompt families.

6.2 Overall Performance

Metric	Value
Test Set Size	3,027
Macro F1 (Full Pipeline)	0.9411
Attack Success Rate (Overall)	1.77%
False Refusal Rate	3.94%
Composite Score	0.9656
ROC AUC (Attack vs. Benign)	0.9859
PR AUC	0.9852
Inference Latency (Mean)	5.836 ms
Inference Throughput	~171 req/s
Total Misclassifications (Threshold-Optimized)	131/3,027 (4.32%)

Table 6.1— Final Performance

6.3 Confusion Matrix Analysis

Actual \ Predicted	**Benign**	**Jailbreak**	**Harmful**
Benign	1171	34	14
Jailbreak	17	1396	41
Harmful	15	11	329

Table 6.2— Confusion Matrix

The matrices show that:

- Most errors are **WRONG_TYPE** (jailbreak ↔ harmful), which are *safe failures*.
- **32 false negatives** contribute to ASR.

- Regex reduces interpretability cost while not increasing ASR.

6.4 Threshold Sweep Analysis

Threshold sensitivity is evaluated using a systematic sweep over 15 candidate thresholds (0.15–0.85).

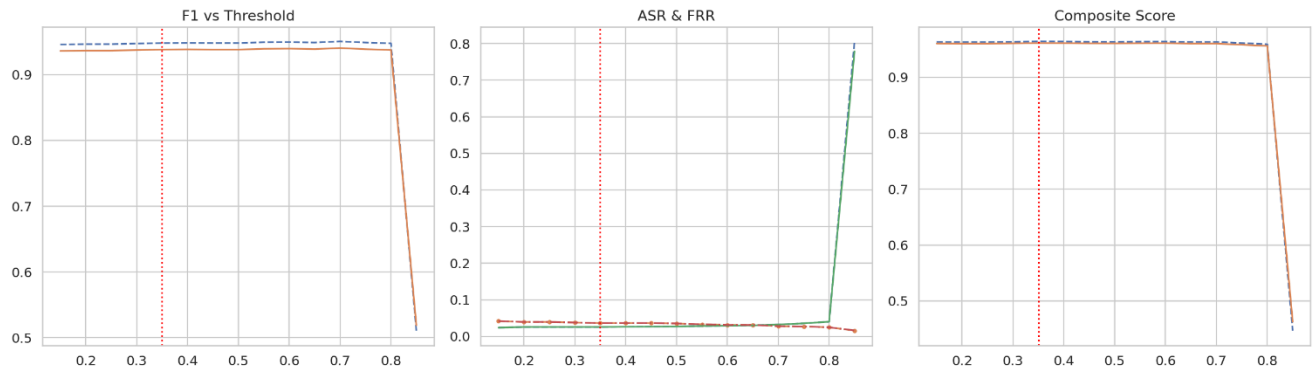

Figure 6.2— Threshold Sweep Analysis and Composite Score

As the threshold increases:

- F1 increases
- FRR decreases
- ASR increases

The composite score plot exhibits a clear optimum, validating the production thresholds.

6.5 Error Taxonomy

Error Type	**Count**	**Interpretation**
False Negative (FN)	32	Safety-critical
False Positive (FP)	48	Usability impact
WRONG_TYPE	51	Safe failure

Table 6.3— Error Taxonomy

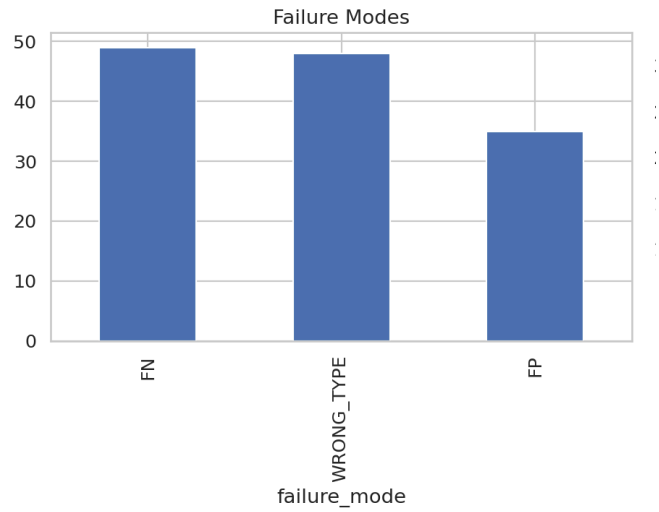

Figure 6.3— Failure modes visualization

6.6 Confidence Distribution and Failure Modes

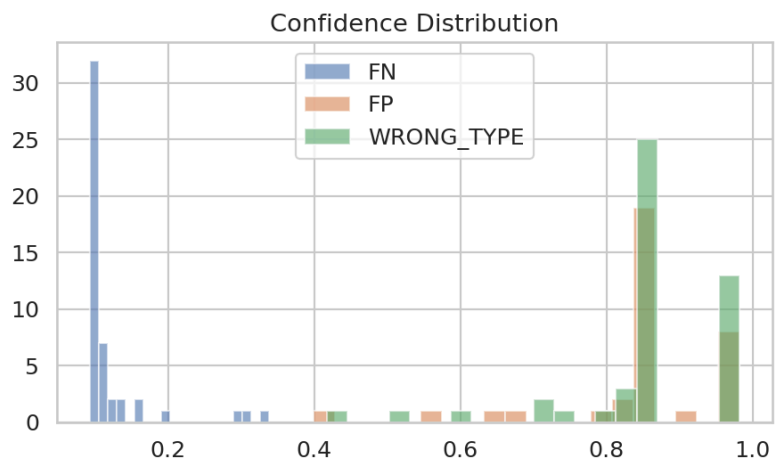

Figure 6.4— Confidence Distribution by Error type

Analysis shows:

- False negatives cluster at **low attack probabilities (< 0.3)**.
- False positives cluster near the **allow/transform boundary**.
- WRONG_TYPE errors correlate strongly with regex hits.

This behavior confirms that errors arise from ambiguity and boundary sensitivity rather than model saturation.

6.7 Model vs. Model + Regex Comparison

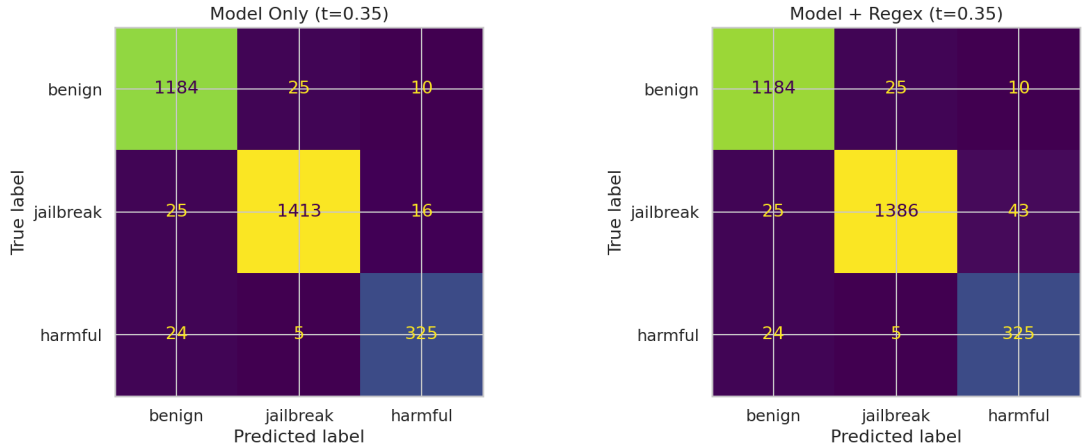

Figure 6.5— Confusion Matrices (Model only vs Model + Regex)

Decision Layer	Proportion	Accuracy
Allow (Pass-through)	~50%	96.18%
Model (Block)	~46%	96.84%
Regex (Block)	~3.5%	46.15%
Transform	<0.5%	0.00%

Table 6.4— Layer Wise Decision Attribution

The regex layer's low standalone accuracy reflects its intended role as a fast, high-recall pre-filter. The neural classifier provides precision, and the transformation layer activates only in rare borderline cases.

6.8 Summary of Evaluation Findings

The evaluation shows that the guardrail system:

- Maintains low ASR while keeping FRR within acceptable bounds
- Benefits from complementary rule-based and neural components
- Enables controllable safety behavior via calibrated thresholding

- Exhibits interpretable, bounded failure modes rather than systemic collapse

7. Deployment

7.1 Hugging Face Hub Repository (Model Deployment Guide)

This section details the process of exporting the trained **mDeBERTa-v3-base** jailbreak classifier from a local PyTorch `state_dict` to a production-ready repository on the Hugging Face Hub.

7.1.1 Repository Structure

The model is hosted at: `DS-AI-Group10/guardrail-mdeberta-v3-jailbreak`

A production-ready model repository must contain:

- **`model.safetensors`**: The weights in the modern, safe, and fast-loading format.
- **`config.json`**: Model architecture configuration.
- **`tokenizer.json`** & **`tokenizer_config.json`**: The vocabulary and preprocessing rules.
- **`README.md`**: The Model Card (benchmarks, license, and usage).

7.1.2 Export & Weight Remapping

Our local model used a custom class (`GuardrailModel`) with specific names for layers (`encoder`, `head`). To make this compatible with the Hugging Face `AutoModelForSequenceClassification` ecosystem, we performed a name remapping during export.

Mapping Logic

Local Name	HF Standard Name
`encoder.deberta.*`	`deberta.*`
`head.weight`	`classifier.weight`
`head.bias`	`classifier.bias`

Export Script (`hf_export.py`)

We used the following logic to package the model:

1. Load `final_model.pt` (the raw `state_dict`).
2. Initialize a standard `DebertaV2ForSequenceClassification` from the `microsoft/mdeberta-v3-base` skeleton.
3. Iterate through the local `state_dict`, rename keys to match the HF skeleton, and load them.
4. Call `model.save_pretrained("./model_package")` to generate the `safetensors` and `config.json`.

7.1.3 Model Card & Metadata

The `README.md` (Model Card) was generated to include:

- **Evaluation Metrics**: 95.67% Accuracy and 0.9656 Composite Score.
- **Library Metadata**: Explicitly tagged with `transformers`, `pytorch`, and `safetensors` to enable the "Inference Widget" on the Hub.
- **License**: MIT.

7.1.4 Security & Safety

- **Safetensors**: We intentionally used the `.safetensors` format instead of `.bin` (Pickle). This prevents code execution vulnerabilities during model loading, which is critical for a security-focused project.
- **Private Secrets**: All API keys LLM used for the companion Inference Space are managed via HF Secrets and are never committed to the repository.

Status: Successfully Deployed

Hub Link: huggingface.co/DS-AI-Group10/guardrail-mdeberta-v3-jailbreak

7.2 Deployment Details (Hugging Face Spaces)

The Guardrail System is deployed as a high-performance web application on **Hugging Face Spaces**, optimized for low-latency CPU inference.

7.2.1 Deployment Method: Cloud (Hugging Face Spaces)

- **SDK**: Gradio (Python-based interactive UI).
- **Environment**: Hugging Face "CPU Basic" (Free Tier).

7.2.2 Access & Endpoints

- **Public URL**: https://huggingface.co/spaces/DS-AI-Group10/gaurdrail_system_demo

7.2.3 Infrastructure & Configuration

The deployment is managed via a `requirements.txt` file and a central `app.py` entry point.

Key Dependencies:

- `transformers`: For mDeBERTa-v3 model loading and tokenization.
- `torch`: Optimized for CPU math and quantization.
- `safetensors`: Ensures secure and fast weight loading.

7.2.4 Hosting Limitations & Constraints

- **Inference Latency**: While the Guardrail itself (Layer 0 & 1) runs in ~10-20ms, the total response time is dependent on the LLM API's network latency during the `TRANSFORM` or `ALLOW` actions.
- **Concurrent Users**: The "CPU Basic" tier supports limited concurrency. For high-traffic production use, upgrading to **ZeroGPU** or **Private Infrastructure** is recommended.
- **Memory Limit**: The current container is limited to 16GB RAM, which is more than sufficient for the 1.1GB mDeBERTa model.
- **Cold Starts**: If the Space has been inactive for 48 hours, it will "sleep." The first request after a sleep period will experience a ~30-second delay for container initialization.

8. Conclusion

This work presents a **production-ready, inference-time guardrail system** for mitigating prompt jailbreak attacks against large language models. The system combines **deterministic regex filtering**, **semantic neural classification using mDeBERTa-v3-base**, **threshold-based tri-modal decision logic**, and an **optional LLM-powered prompt transformation stage**, forming a hybrid defense-in-depth architecture.

Evaluation on a held-out test set of **3,027 prompts** demonstrates that the system achieves a strong balance between safety and usability, with an **Attack Success Rate (ASR) of 1.77%**, a **False Refusal Rate (FRR) of 3.94%**, and a **Macro-F1 score of 0.9411** under production thresholds. The end-to-end inference latency of approximately **5.84 ms per request** on a T4 GPU confirms that the system is suitable for real-time deployment.

Confusion-matrix analysis comparing **model-only** and **model + regex** configurations shows that the majority of misclassifications fall into the **WRONG_TYPE category (jailbreak ↔ harmful)**, which are *safe failures* since both classes result in a BLOCK action. The number of **false negatives (32)** directly accounts for ASR contributions, while the inclusion of the regex layer does **not increase ASR**, but provides a fast, interpretable first-line filter for well-known attack patterns.

Threshold sweep analysis further highlights the controllable safety-utility trade-off: as thresholds increase, **F1 improves**, **FRR decreases**, and **ASR increases**, with the composite score exhibiting a clear optimum that validates the chosen production thresholds. Confidence-distribution analysis reveals that most failures arise from **boundary sensitivity and semantic ambiguity**, rather than systemic model collapse or overconfidence.

Overall, the results confirm that **hybrid architectures outperform single-layer systems** in safety-critical LLM deployments. The combination of rules, learned semantics, and calibrated decision logic yields a guardrail that is interpretable, efficient, and robust to a wide range of adversarial prompt strategies.

9. Future Work and Limitations

9.1 Identified Limitations

1. **Out-of-Distribution (OOD) Generalization**: The 30-prompt OOD demonstration set exhibits a higher ASR (25%) than the in-distribution test set. These prompts are intentionally short, ambiguous, and lack the structural markers present in the training distribution. While this behavior exposes meaningful decision boundaries rather than representing a production failure, it highlights the need for larger-scale OOD evaluation.
2. **Harmful Class Performance**: The harmful class shows the lowest F1 score (0.8916) and the highest per-class ASR under threshold-optimized evaluation (4.24%). This reflects both its smaller representation in training data and the difficulty of detecting short, direct harmful requests that lack explicit adversarial framing.

3. **Transformation Quality Evaluation**: The LLM-based transformation stage is currently evaluated only through re-classification safety checks. There is no formal assessment of intent preservation, semantic drift, or linguistic quality of the transformed prompts.
4. **Language Coverage**: Both the regex rules and training data are English-only. Prompts written in other languages or multilingual mixtures may bypass the regex layer and be inconsistently handled by the classifier.
5. **Static Rule Set**: The regex pre-filter relies on a manually curated set of patterns, which must be periodically updated as jailbreak strategies evolve.

9.2 Proposed Future Work

1. **Expanded OOD Evaluation**: Assemble a larger OOD benchmark (200–500 prompts per category) from red-teaming datasets, real-world user logs (with appropriate privacy protections), and adversarial prompt-generation tools to obtain statistically meaningful ASR and FRR estimates.
2. **Harmful-Class Data Augmentation**: Augment the training set with additional short-form harmful prompts to reduce class imbalance and improve recall for concise harmful requests.
3. **Adaptive Regex Rule Updates**: Develop a feedback-driven mechanism that uses error-analysis outputs to identify common evasion patterns and update regex rules accordingly.
4. **Multilingual Extension**: Extend both the regex patterns and training corpus to cover additional languages commonly used in adversarial attacks, improving global robustness.
5. **Human-in-the-Loop Transformation Evaluation**: Introduce human annotation to assess LLM transformation quality along dimensions of **intent preservation**, **safety**, and **fluency**, enabling more rigorous evaluation of the transform stage.

10. References

1. JailbreakBench: Structured adversarial categories for LLM safety.
2. He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with disentangled attention*.
3. microsoft/mdeberta-v3-base: Encoder-only transformer for sequence classification.

4. OWASP Top 10 for LLM Applications: Guidance on prompt-injection mitigation.
5. Bergstra, J. & Bengio, Y. (2012). *Random search for hyper-parameter optimization*.
6. Google Generative AI Documentation: Gemini API reference and usage guidelines.

11. Appendix: Figures and Tables Index

This appendix groups all figures and tables referenced in the report by their corresponding major sections. Figure and table numbering matches usage in the main text.

Appendix A: Section 3 - Dataset

Tables

- **Table 3.1** – Class Distribution
- **Table 3.2** – Prompt-to-Family Cardinality Analysis
- **Table 3.3** – Jailbreak Taxonomy

Figures

- **Figure 3.1** – Prompt Class Distribution
- **Figure 3.2** – Dataset Split Distribution
- **Figure 3.3** – Class Distribution Across Splits
- **Figure 3.4** – Top Data Sources
- **Figure 3.5** – Prompt Length Distribution by Class
- **Figure 3.6** – Jailbreak Strategy Frequency

Appendix B: Section 4 — Methodology

Figures

- **Figure 4.1** – Inference-Time Guardrail Middleware Architecture

Appendix C: Section 5 — Model Development and Hyperparameter Tuning

Figures

- **Figure 5.1** – Model Architecture
- **Figure 5.2** – Token Length Distribution
- **Figure 5.3** – Population Coverage vs. Truncation Point

Tables

- **Table 5.2** – Sample Coverage Based on Sequence Length

Appendix D: Section 6 — Evaluation and Analysis

Figures

- **Figure 6.2** – Threshold Sweep and Composite Score
- **Figure 6.3** – Failure Modes Visualization
- **Figure 6.4** – Confidence Distribution by Error Type
- **Figure 6.5** – Confusion Matrices (Model-Only vs. Model + Regex)

Tables

- **Table 6.1** – Final Performance
- **Table 6.2** – Confusion Matrix
- **Table 6.3** – Error Taxonomy
- **Table 6.4** – Layer-Wise Decision Attribution